

Decision Trail: From ARM64 Incompatibility to Working NEON Simulations

Period: May-June 2026 **Author:** Steven Wangen, UW-Madison Data Science Institute

This document traces the full chain of decisions and discoveries from the initial problem (container doesn't run on Apple Silicon) through the current state (CTSM container with NEON workflow, blocked on a data configuration issue with a known fix). Each section describes what we knew at the time, what we decided, and what that led to next.

1. The starting problem: amd64 container on arm64 hardware

What we had: A Docker container based on `escomp/cesm-lab-neon`, published by NCAR around 2022. It ran CESM 2.2 with the NEON tower site workflow, shipped Python 3.7, and was built exclusively for Intel/AMD processors (amd64).

What broke: Team members on Apple Silicon Macs (M1-M4) had to run the container through QEMU emulation, which translated Intel instructions to ARM on the fly. This was 2-3x slower than native execution and caused frequent kernel crashes, particularly during computationally intensive operations like cartopy map rendering and Fortran compilation. Local development was effectively unusable.

Decision: Rebuild the container from scratch with native arm64 support.

Reference: [ADR-0001](#) (base image), [ADR-0002](#) (build strategy)

2. Choosing a new base image

Options evaluated: Ubuntu 24.04, AlmaLinux 9, Rocky Linux 9, Debian 12, jupyter/scipy-notebook, condaforge/miniforge3.

What we learned: Both Pangeo and Jupyter Docker Stacks have standardized on Ubuntu 24.04 for scientific Python containers. It has the longest free support (2029), the smallest base (~29 MB), native arm64 manifests, and the best-documented path for Fortran/C compilation.

Decision: Ubuntu 24.04 LTS.

What this led to: All `yum / dnf` commands from the old CentOS image needed translation to `apt-get`, but the scope was smaller than expected because most scientific libraries moved to conda-forge (see next step).

Reference: [ADR-0001](#)

3. Eliminating source compilation

What we had: The upstream image hand-compiled MPICH 3.3.2, HDF5 1.12.0, NetCDF-C 4.7.4, NetCDF-Fortran 4.5.3, and PNetCDF 1.12.1 from source. This took 30-45 minutes and was the primary source of architecture-specific build failures.

What we learned: All of these libraries are available as pre-built conda-forge binaries for both amd64 and arm64. Using conda-forge also ensures ABI consistency across the MPI stack (MPICH, mpi4py, ESMF, NetCDF all from the same build chain).

Decision: Replace source compilation with conda-forge binaries. Use conda-lock for reproducible cross-platform lockfiles.

What this led to: Build time dropped from ~45 minutes to ~5 minutes. Eliminated the entire "compile C/Fortran libraries" stage of the Dockerfile.

Reference: [ADR-0003](#), [ADR-0004](#)

4. Thirteen compatibility fixes

What happened: Building CESM 2.2.2 with modern conda-forge toolchain (GCC 15, NetCDF-C 4.9+, Python 3.11+) on arm64 exposed 13 separate compatibility issues. Each was discovered through the automated test suite and fixed iteratively.

Key fixes: - `rdtsc` x86 assembly in GPTL timer (arm64 has no `rdtsc` instruction) - GCC 10+ strict type checking (`-fallow-argument-mismatch` , `-fallow-invalid-boz`) - NetCDF-C macro rename (`_FillValue` to `NC_FillValue`) - PIO2 filter API mismatch (newer NetCDF headers advertise functions that old PIO2 doesn't implement) - Python 3.12 `import imp` removal (CIME 2.2.x used the deprecated `imp` module) - cmake 4.x compatibility (CESM's PIO used `cmake_minimum_required(VERSION 3.0.2)`) - Bundled `six.py` shadowing conda-forge version (broke `dateutil`)

What this led to: A 90-test validation suite across three tiers (smoke, case creation, build + analysis). All 90 tests passing on native arm64.

Reference: [ADR-0002 Implementation Notes](#), test suite in `tests/`

5. Discovering that NEON support was never in CESM 2.x

What we expected: The old container had NEON site support, so CESM 2.2.2 should too.

What we found: `run_neon_v2 --neon-sites ABBY` failed with "invalid choice: 'ABBY' (choose from 'all')". The NEON usermods directory did not exist in CESM 2.2.2's CLM checkout.

What the research showed: The NEON tower workflow was developed on CTSM's master branch starting February 2021 (PR #1278), well after the CESM 2.2 release line was cut. No CESM 2.x release has ever included NEON support. The original `escomp/cesm-lab-neon` Docker image was a custom, undocumented build that grafted a newer CTSM development branch onto a CESM 2.2 framework. NEON support will arrive in CESM when version 3.x ships (currently in beta, spring 2026 target slipped).

Decision: Replace the full CESM framework with standalone CTSM.

What this led to: ADR-0005 and the evaluation of CTSM versions.

Reference: [Decision brief 001](#), [ADR-0005](#)

6. Choosing standalone CTSM over full CESM

Why CTSM: NCAR designed and maintains the NEON workflow as a CTSM-standalone feature. The project runs CLM in I-compset mode (land-only with data atmosphere), which is exactly what standalone CTSM is designed for. The full CESM components (CAM, POP, CICE, CISM, WW3) were present in the container but never compiled or executed.

Trade-off: Losing the ability to run fully coupled atmosphere-land-ocean simulations. The project does not currently use coupled simulations; all workflows run CLM against observed forcing data. If coupled experiments are needed in the future, a separate CESM 3.x container can be maintained.

What this led to: A cleaner, smaller container with NEON support natively included.

Reference: [ADR-0005](#)

7. CTSM 5.4 vs 5.2: both blocked, for different reasons

CTSM 5.4.002 (December 2025): Container builds cleanly, 90/90 tests pass, NEON sites discoverable, case.build produces cesm.exe. But live simulation fails: input data download returns 404 for several files.

CTSM 5.2.005 (August 2024): Expected to have established input data. But its older CIME has a Python 3.13 XML parsing incompatibility (`_Element` vs `Element` type error). Would require pinning Python back to 3.11–3.12, undoing work from the rebuild.

Decision: Stay on CTSM 5.4. Its blocker (missing data) is a configuration issue that can be resolved. CTSM 5.2's blocker (Python incompatibility) would require regressions.

Reference: [Decision brief 002](#)

8. Diagnosing the data availability issue

Initial hypothesis: NCAR hadn't published input data for the 5.4 release. Six months after release seemed surprising but plausible.

Server probing: Tested individual file URLs across NCAR's FTP, SVN, and NEON-specific servers. The NEON surface dataset

(`surfdata_1x1_NEON_KONZ_hist_2000_78pfts_c251023.nc`) returned 404 on all servers. However, the NEON server hosted pre-computed transient run output for 45 sites (daily history files, 2018-2022), suggesting the data pipeline works internally but public serving is incomplete.

Release notes analysis: Found the `CLM_CMIP_ERA` flag (cmip7 vs cmip6) and the statement that CMIP7 data "are only available through the historical record (1850-2023), and are not available for future periods (presently known as SSP)."

NEON usermods inspection: The NEON defaults set `DATM_PRESAERO=SSP3-7.0` (and similar) because NEON runs cover 2018-2021, extending past the historical period. But the IHist compset is not technically an SSP compset, so `CLM_CMIP_ERA` defaults to `cmip7` instead of `cmip6`.

Reference: [Decision trail 003](#)

9. Root cause: CLM_CMIP_ERA mismatch

Deep research verification: Multi-source search confirmed: - No one has successfully run NEON simulations with ctsm5.4.002 in Docker - The `escomp/ctsm-neon` Docker image unchanged since October 2022 - NCAR's CTSM tutorial skipped 5.4, runs on internal cloud with pre-staged data - The EMBER Tutorial 2025 (May 2025) jumped from CLM5.1 to CLM6.0 - The `CLM_CMIP_ERA` auto-detection does not account for NEON IHist compsets using SSP-era forcing dates

Root cause: The NEON IHist compset triggers `CLM_CMIP_ERA=cmip7` (because it's not an SSP compset), but the DATM settings request SSP3-7.0 forcing dates, causing the namelist generator to look for CMIP7-era SSP files that have not been produced.

Proposed fix: Set `CLM_CMIP_ERA=cmip6` explicitly for NEON runs. The release notes say SSP-period data should use CMIP6. This is the intended behavior; the auto-detection just doesn't cover the NEON case.

Status: Fix identified, not yet tested.

Reference: [ADR-0006](#)

Current state

What works: - Multi-arch container (amd64 + arm64) on Ubuntu 24.04 - CTSM 5.4 with 48 NEON tower sites - Python 3.13, conda-forge scientific stack - 90/90 test suite passing (imports, compilers, case creation, Fortran build, analysis workflows) - JupyterLab with project notebooks - Getting Started notebook (no credentials needed)

What is blocked: - Running a live CTSM simulation (proposed fix: `CLM_CMIP_ERA=cmip6` , not yet tested) - Modeling_Hub notebook (needs either simulation output or NCAR's pre-computed data)

Next step: Test the `CLM_CMIP_ERA=cmip6` fix. If it resolves the data download, the full simulation pipeline becomes functional and the Modeling_Hub notebook can run end-to-end.